

AALeC Quiz

Multiplayer-Quizsystem mit Embedded Hardware-Controllern



Pascal May
68763

Florian Merlau
81775

Projektidee & Anwendungsfall

— HOCHSCHULE AALEN · TECHNISCHE INTERNETBASIERTE SYSTEME · SS 2026

AALeC Quiz

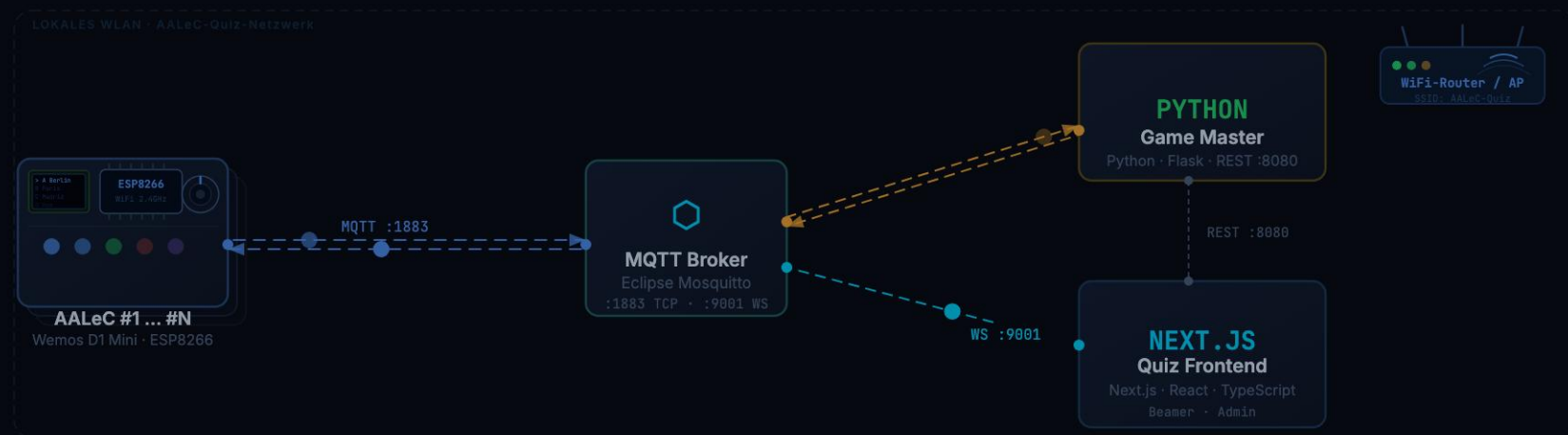
Multiplayer-Quizsystem mit Embedded Hardware-Controllern



Pascal May
68763

Florian Merlau
81775

Vom Controller bis zum Beamer



AALeC-V3 — Wemos D1 Mini (ESP8266)

KOMPONENTE	FUNKTION IM QUIZ
OLED SH1106	Frage + A/B/C/D Antworten
Drehgeber + Taster	Auswahl bestätigen
WS2812B LEDs × 5	Richtig = grün, Falsch = rot
Potentiometer A0	Schätz- & Poti-Challenge
BME680 Sensor	Temperatur-Challenge
Piezo Beeper	Sound-Feedback
WiFi 802.11 b/g/n	MQTT über TCP/WS



PYTHON
Game Master

Python Flask / REST API

REST API

NEXT.JS
Quiz Frontend

React, TypeScript
Redux, Axios

GPIO-Belegungsplan D1 Mini (ESP8266)

GPIO	PIN-NAME	KOMPONENTE & FUNKTION
GPIO 0	D3	Taster → Antwort bestätigen
GPIO 2	D4	WS2812B RGB-LEDs
GPIO 4	D2	I ² C SDA ↔ OLED
GPIO 5	D1	I ² C SCL ↔ OLED
GPIO 12	D6	Rotary Encoder Kanal A
GPIO 14	D5	Rotary Encoder Kanal B
GPIO 15	D8	Piezo-Beeper (PWM)
A0 (ADC)	A0	Potentiometer (10-Bit)
WIFI 802.11 b/g/n		MQTT über TCP/WS

I²C

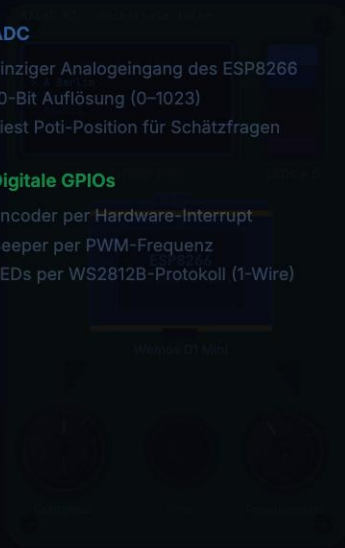
2-Draht-Bus (SDA + SCL)
OLED SH1106 · Adresse 0x3C
BME680 Sensor · Adresse 0x76

ADC

Einzigster Analogeingang des ESP8266
10-Bit Auflösung (0-1023)
Liest Poti-Position für Schätzfragen

Digitale GPIOs

Encoder per Hardware-Interrupt
Beeper per PWM-Frequenz
LEDs per WS2812B-Protokoll (1-Wire)



Display-Zustände & UI-Konzept



Code-Aufbau & Software-Architektur



Verwendete Bibliotheken-Architektur

C++ Firmware

PubSubClient MQTT-Client MQTT-Kommunikation

ArduinoJson JSON-Serialisierung JSON serialisieren / parsen

ESP8266WiFi WLAN-Verbindung WLAN-Verbindung

SSD1306Wire OLED-Display (I²C) OLED-Display (I²C)

Adafruit_BME680 Temperatursensor Temperatursensor

NeoPixelBus WS2812B RGB-LEDs WS2812B RGB-LEDs

MQTTClient MQTT-Kommunikation MQTT-Kommunikation

Py Backend Master / API

Flask REST-API REST-API

paho-mqtt MQTT-Client (Game Master) MQTT-Client (Game Master)

python-dotenv Konfiguration via .env Konfiguration via .env

scoring.py Punkte + Basis (1000) + Zeitbonus (max. 500) * Streak-Bonus

app.py Flask API

mqtt.js MQTT-Client (Game Master)

mqtt.js MQTT-Client (Game Master)

TS Frontend

Next.js / React UI-Framework UI-Framework

mqtt.js MQTT über WebSocket (Port 9001) MQTT über WebSocket (Port 9001)

Tailwind CSS Styling Styling

Admin-Panel (/admin)

mqtt.js MQTT über WebSocket (Port 9001)

mqtt.js MQTT über WebSocket (Port 9001)

mqtt.js MQTT über WebSocket (Port 9001)

MQTT-Kommunikationen

SERVER → ALLE

C++ Firmware

PubSubClient	MQTT-Kommunikation
ArduinoJson	JSON von Arduino Ethernet
ESP8266WiFi	WiFi-Verbindung
SSD1306Wire	Displaylibrary I2C
Adafruit_DME688	Temperatursensor
NeoPixelBus	RGB-LEDs ansteuern

CLIENTS → SERVER

Py Backend

Flask	Webserver
paho-mqtt	MQTT Client (Arduino Mosquitto)
python-dotenv	Umweltvariable laden



Mosquitto

:1883
TCP
ESP8266

:9001
WS
Browser

JS Frontend

Next.js / React	Webbrowser
mqtt.js	MQTT Client Webbrowser (Next.js)
Tailwind CSS	Styling

Device-ID: **aAlec<hex>**
aus ESP8266 Chip-ID generiert



MQTT Broker: Eclipse Mosquitto in Docker

SERVER: ECHO



Eclipse Mosquitto 2

Open-Source MQTT Broker · läuft als Docker-Container

- Image: `eclipse-mosquitto:2` · direkt von Docker Hub
- Config via Volume-Mount: `mosquitto.conf` read-only eingehängt
- Healthcheck per `mosquitto_sub` — Game Master startet erst wenn Broker bereit

ZWEI LISTENER

TCP :1883 ESP8266 Firmware via PubSubClient

WS :9001 Browser-Frontend via mqtt.js (WebSocket)

CONFIG: mosquitto.conf SERVER

```
# TCP für ESP8266 Firmware listener 1883 protocol mqtt #
WebSocket für Browser-Frontend listener 9001 protocol websockets
allow_anonymous true
```

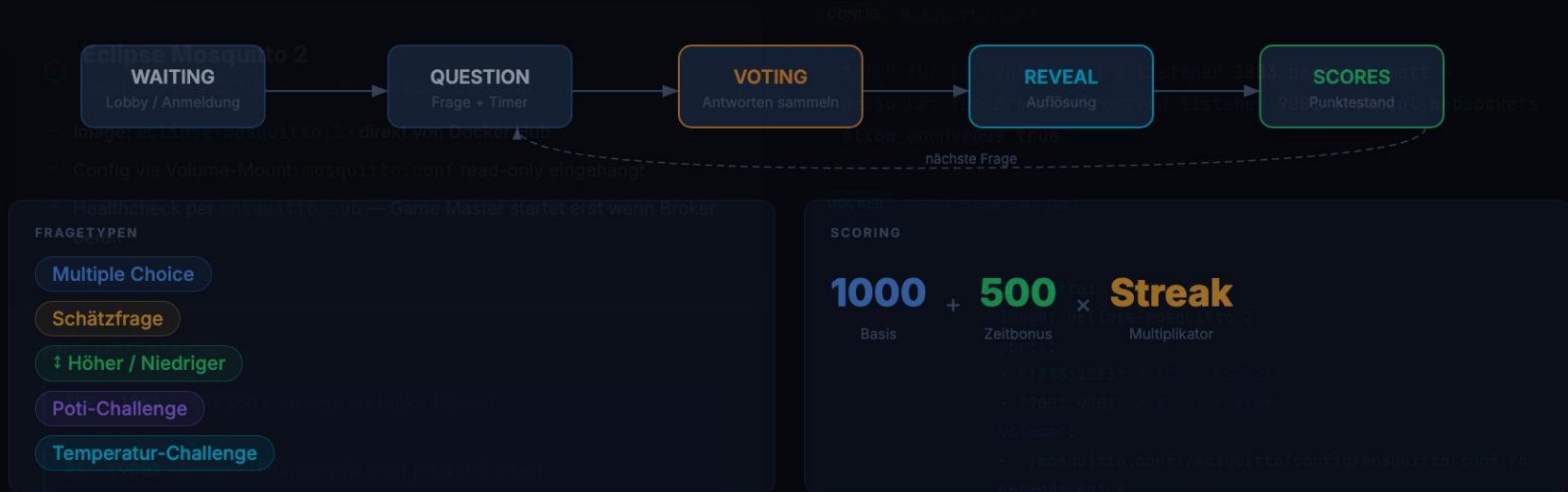
Mosquitto

DOCKER: docker-compose.yml

1883 : 1883

```
mosquitto:
  image: eclipse-mosquitto:2
  ports:
    - "1883:1883" # TCP → ESP8266
    - "9001:9001" # WS → Browser
  volumes:
    - ./mosquitto.conf:/mosquitto/config/mosquitto.conf:ro
  depends_on: #
    game-master wartet auf healthy
  healthcheck:
    test: mosquitto_sub -t
      $$SYS/# -C 1 -W 3
    interval: 10s
    timeout: 5s
    retries: 3
```

Backend — Game Master & State Machine



Fragetypen & Datenschema State Machine

mcq - Multiple Choice

```

{
  "type": "mcq",
  "WAITING": {
    "text": "Idle-Pegel einer UART-Leitung?",
    "options": {
      "A": "LOW", "B": "HIGH",
      "C": "floating", "D": "Takt"
    },
    "correct": "B", "time_limit_s": 20
  },
  "QUESTION": {
    "text": "Welche Antwort ist richtig?",
    "options": {
      "A": "A", "B": "B", "C": "C", "D": "D"
    }
  },
  "VOTING": {
    "text": "Bitte eine Antwort auswählen!"
  }
}

```

Multiple Choice

Schätzfrage

Höher / Niedriger

estimate - Zahlenwert schätzen

```

{
  "type": "estimate",
  "REVEAL": {
    "text": "millis()-Überlauf nach  
wie vielen Tagen?",
    "min": 1, "max": 100,
    "correct": 50, "unit": "Tage"
  },
  "SCORES": {
    "text": "Deine Punktzahl ist: 1000"
  }
}

```

1000

Punkte

500

Zählerlauf

Streak

Kontinuität

poti_challenge

higher_lower - Höher oder niedriger?

Temperatur-Challenge

```

{
  "type": "higher_lower",
  "text": "RS485-Reichweite > 100 m?",
  "reference": 100,
  "actual": 1200,
  "correct": "HIGHER", "unit": "m"
}

```

poti_target / temp_target - Hardware

```

{
  "type": "poti_target",
  "text": "Poti auf Zielwert einstellen!",
  "target_range": [15, 90], // zufällig
  "tolerance": 5,
  "time_limit_s": 25
}

```

Beamer-View — Was der Projektor zeigt

1 — WARTEN

Nullstelle-Klasse

localhost:3888/

```

{
  "text": "Idle-Paget einer UART-Leitung?",
  "options": {
    "A": "Idle", "B": "Right",
    "C": "floating", "D": "Takt"
  }
}
connect: "B", "time_limit_s": 20

```

AALeC Quiz

Warte auf Geräte...

Noch keine Geräte verbunden.
Noch 1 Spieler fehlt zum Starten.

Quiz starten

2 — FRAGE LÄUFT

localhost:3888/

```

estimate — Zahlenwert schätzen
{
  "type": "estimate",
  "text": "millis()-Überlauf nach  
wie vielen Tagen?",
  "min": 1, "max": 100,
  "correct": 50, "unit": "Tage"
}

```

Frage 3 / 16

12s

Welche Programmiersprache wird für die ESP8266-Firmware eingesetzt?

A C++

B Python

C Java

D Rust

3 / 8 Antworten eingegangen

3 — AUFLÖSUNG

localhost:3888/

```

{
  "type": "estimate",
  "text": "millis()-Überlauf nach  
wie vielen Tagen?",
  "min": 1, "max": 100,
  "correct": 50, "unit": "Tage"
}

```

RICHTIG! ✓

Richtige Antwort: A — C++

A	55%
B	20%
C	15%
D	10%

Leaderboard wird aktualisiert...

1. BAABA — 1.200 Pkt

2. ANNA — 900 Pkt

1 / 21

Admin-Panel — Spielleiter-Steuerung

localhost:8080/admin

1 — FRAGE LAUFT

1 — AUFLÖSUNG

Quiz Admin

http://localhost:8080

MARKT

Starten

Abmelden

Neustart

TEILNEHMER (3)

BAABA
online

ANNA
online

LEO
online

FRAGEN-SETS

▼ questions

questions.json

Exportieren

Multiple Choice

In welchem WLAN-Hotspot-Netzwerk müssen sich die AALeC-Clients anmelden?

Multiple Choice

Welcher Programmbereich wird genau einmal beim Starten des Controllers ausgeführt?

Multiple Choice

Was ist das Hauptterminal des 'Superloop'-Modells (Arduino standard loop)?

12b

Schätzfrage

Wie viele Leitungen werden für die bidirektionale serielle UART-Kommunikation benötigt?

Welche Programmiersprache wird für die bidirektionale serielle UART-Kommunikation eingesetzt?

Poll-Challenge

Welche Sensor-Schnittstelle nutzt typischerweise eine Brückenschaltung?

Multiple Choice

Was passiert beim ESP8266, wenn eine blockierende Funktion länger als ein paar Sekunden läuft?

Multiple Choice

Was unterscheidet ein Thermoelement von einem NTC-Widerstand?

RICHTIG! ✓

Richtige Antwort: A — C++

A

85%

B

20%

C

75%

D

20%

Legende wird aktualisiert

1. BAABA — 1.200 Pkt

2. ANNA — 300 Pkt

Warte auf Gäste

Noch keine Gäste verbunden

Noch 1 Spieler fehlt zum Starten

Quiz starten

1 / 21

1

2

3

4

5

6

7

8

9

10

11

12

13

14

15

16

17

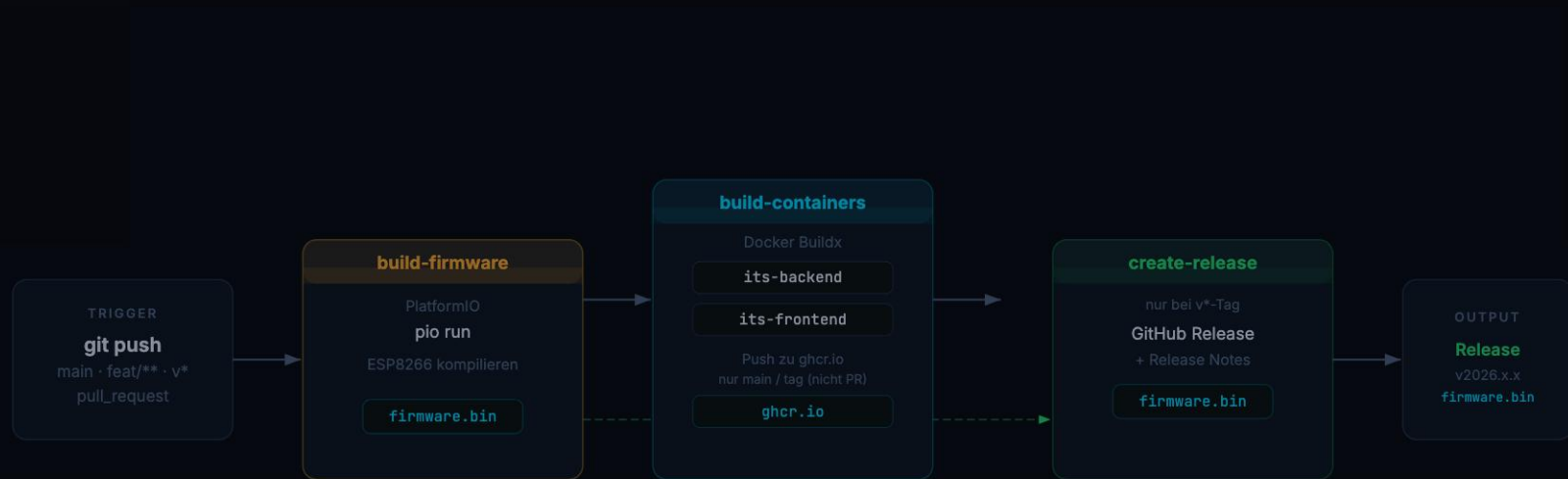
18

19

20

21

GitHub Actions — Build, Push, Release



GitHub Actions — Build, Push, Release

LIVE

Demo

Gemeinsam installieren & Multiplayer testen

📁 Firmware flashen

👤 Multiplayer starten

📈 Quiz spielen



Projekt einrichten & starten

1 — Konfiguration

Firmware — `config.h`

```
// WLAN
#define WIFI_SSID "AALeC-Quiz"
#define WIFI_PASSWORD "12345678"

// MQTT-Broker
#define MQTT_BROKER "MacBook.local"
```

Backend — `.env`

```
MQTT_HOST=localhost
MQTT_PORT=1883
```

2 — Server starten

MQTT-Broker, Backend und Frontend starten:

```
docker-compose up -d
```

Startet drei Container:

<code>mosquitto</code>	MQTT-Broker :1883 / :9001
<code>game-master</code>	Backend / API :8080
<code>frontend</code>	Web-UI :3000

3 — Netzwerk

Mac als WLAN-Hotspot mit fester SSID:

SSID `AALeC-Quiz`

Passwort `12345678`

Broker `MacBook-Pro-von-Florian.local`

Controller verbinden sich automatisch sobald WLAN und Broker erreichbar sind.

Gemeinsam installieren & Multiplayer testen

1. Firmware flashen

2. Multiplayer starten

3. Quiz spielen



Firmware flashen & starten

flash.py — lädt Firmware automatisch von GitHub und erkennt den USB-Port

1 — Konfiguration

```
# Repo klonen
git clone https://github.com/XplorodoX/ITS.git
cd ITS
# Abhängigkeiten installieren
pip install esptool

# Flashen
python3 flash.py

MQTT_HOST=localhost
MQTT_PORT=1883
```

2 — Server starten

MQTT-Broker, Backend und Frontend starten:

```
docker-compose up -d
```

Startet drei Container:

mosquitto

MQTT-Broker: 1883 / 9001

game-master

Backend / API: 8080

frontend

Web-UI: 13000

3 — Netzwerk

Mac als WLAN-Hotspot mit fester SSID:

SSID: **AALeC-Quiz**

Password: **12345678**

Broker: **MacBook-Pro-von-Florian.local**

Controller verbindet sich automatisch sobald WLAN und Broker erreichbar sind.

Browser (kein Python nötig)

firmware.bin von GitHub Releases laden, dann im Browser flashen:

esptool.spacehuhn.com

Manuell (Terminal)

firmware.bin von GitHub Releases laden, dann:

```
# macOS / Linux
python3 -m esptool --port [port] --baud 115200 write_flash 0x0 firmware.bin

# Windows
python -m esptool --port COM3 --baud 115200 write_flash 0x0 firmware.bin
```

Firmware flashen

flash.py — lädt Firmware automatisch von GitHub und erkennt den USB-Port

```
# Repo klonen
git clone https://github.com/Xpl0red0x/ITS.git
cd ITS

# Abhängigkeit installieren
pip install esptool

# Flashen
python3 flash.py
```

AALeC Quiz

Warte auf Geräte...

Noch keine Geräte verbunden.

Manuell (Terminal)

Noch 1 Spieler fehlt zum Starten.

firmware.bin von GitHub Releases laden, dann:

Quiz starten

MacOS / Linux

```
python3 -m esptool --port [port] --baud 115200 write_flash 0x0 firmware.bin
```

Windows

```
python -m esptool --port COM3 --baud 115200 write_flash 0x0 firmware.bin
```

Browser (kein Python nötig)

firmware.bin von GitHub Releases laden, dann im Browser flashen:

esptool.spacehuhn.com

TEILNEHMER (1)

Bro
aaLec-082816

FRAGEN-SETS

questions 18

← Set auswählen oder neu erstellen

AALeC Quiz

Warte auf Geräte...

Noch keine Geräte verbunden.

Noch 1 Spieler fehlt zum Starten.

QUIZ STARTEN

neues-set

+ Set



Quiz Was als Nächstes kommen könnte

TEILNEHMER (1)

• Bro
100% 0/10

FRAGEN-SETS

• questions 100% 0/10

← Set auswählen oder neu erstellen



01

Persistenz & Highscores

Datenbank für Spielhistorie, Highscores und Auswertung pro Spieler über mehrere Sessions



02

Fragen-Editor im Admin-Panel

Fragen direkt im Browser erstellen und bearbeiten statt JSON-Datei manuell zu pflegen



03

Anti-Cheat

Mehrfachabstimmung und Zeitmanipulation serverseitig erkennen und verhindern



04

Bilderfragen

Bilder direkt zur Frage auf dem Beamer anzeigen für visuellere Fragetypen



Was als Nächstes kommen könnte



Fragen



11

Persistenz & Highscores

Datenbank für Spielstatistik, Highscores und Auswertung pro Spieler über mehrere Sessions



17

Fragen-Editor im Admin-Panel

Fragen direkt im Browser erstellen und bearbeiten, statt JSON-Datei manuell zu pflegen



19

Anti-Cheat

Mehrfachabschirmung und Zeitmanipulation serverseitig erkennen und verhindern



20

Bilderfragen

Bilder direkt zur Frage auf dem Desktop anzeigen für visuelle Fragetypen



Fragen

Verwendete Tools & Ressourcen

HARDWARE & FIRMWARE

AALeC-V3 Hardware-Plattform	github.com/informatik-aalen/AALeC-V3
ESP8266 / Wemos D1 Mini	espressif.com/en/products/socs/esp8266
PlatformIO IDE	platformio.org
esptool.py (Flashen)	github.com/espressif/esptool
ArduinoJson	arduinojson.org

BACKEND, BROKER & FRONTEND

Eclipse Mosquitto	mosquitto.org
Python / Flask	flask.palletsprojects.com
Next.js / React	nextjs.org
MQTT Protokoll	mqtt.org
Projekt-Repository	github.com/XplorodoX/ITS

Danke für eure
Aufmerksamkeit

PDF exportieren:

`python3 docs/präsentation/export_pdf.py`